

HW7 Optimal FIR design

Assume cut-off frequency $\omega_c = (\omega_p + \omega_s)/2$

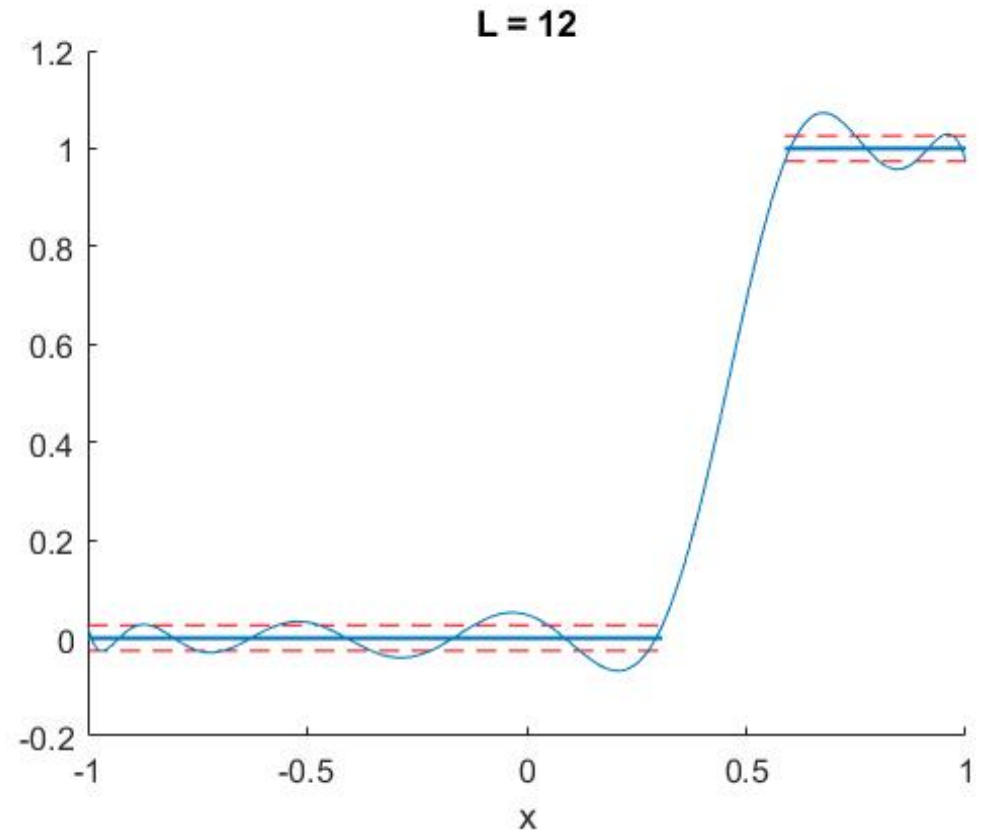
1.

Once ω_p, ω_c at passband and stopband frequencies and the length L are set, it's further required to solve the coefficients for the polynomial denoted as y . Note that tolerance δ is unknown, so it is also to be solved in y . The alternating point x_j consisting matrix A are averagely divided, and the associated value $P(x_j)$ needs to achieve negative and positive tolerance value δ sequentially in the last column of A . Besides, in passband, $P(x_j)$ instead passes through $1 + \delta$ and $1 - \delta$ in order. Hence, vector b implies which terms are located in stopband or passband. In details, the ind -th and $(ind-1)$ -th alternating points are required to pass through ω_p and ω_s , so the corresponding δ must be of different signs. Finally, alternating points are mapped from $\mathbb{R}$ to $\mathbb{P}$.

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^L & -1 \\ \vdots & & & & & \vdots \\ 1 & x_{L+2} & x_{L+2}^2 & \cdots & x_{L+2}^L & (-1)^{L+2} \end{bmatrix}$$

$$y = \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_L \\ \delta \end{bmatrix}$$

$$b = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$



solve y to obtain initial coefficients and tolerance:

$$y = A^{-1}b$$

```
pow = 0:(L+1); % power set from 0 to L
last_column = (-1).^(pow+1); % last column of A consists of -1 and 1
A = xx.^pow(1:end-1); % other columns except the last one have x_ij^(jj-1)
A = [A last_column']; % add last column
```

2.

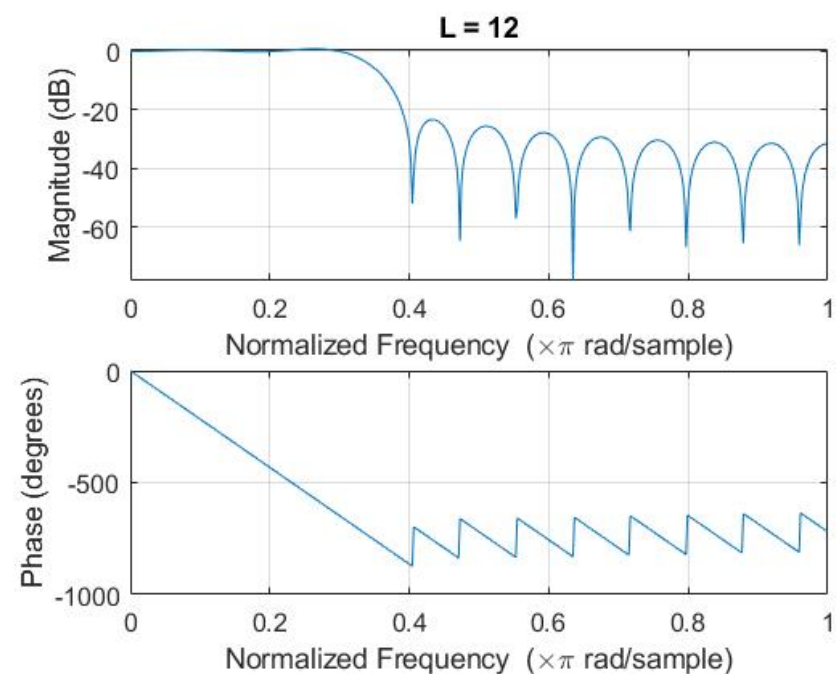
For $L = 12$, the corresponding Chebyshev coefficients are

$$W = \begin{bmatrix} 1 & 0 & -1 & 0 & 1 & 0 & -1 & 0 & 1 & 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & -3 & 0 & 5 & 0 & -7 & 0 & 9 & 0 & -11 & 0 \\ 0 & 0 & 2 & 0 & -8 & 0 & 18 & 0 & -32 & 0 & 50 & 0 & -72 \\ 0 & 0 & 0 & 4 & 0 & -20 & 0 & 56 & 0 & -120 & 0 & 220 & 0 \\ 0 & 0 & 0 & 0 & 8 & 0 & -48 & 0 & 160 & 0 & -400 & 0 & 840 \\ 0 & 0 & 0 & 0 & 0 & 16 & 0 & -112 & 0 & 432 & 0 & -1232 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 32 & 0 & -256 & 0 & 1120 & 0 & -3584 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 64 & 0 & -576 & 0 & 2816 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 128 & 0 & -1280 & 0 & 6912 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 256 & 0 & -2816 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 512 & 0 & -6144 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1024 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2048 \end{bmatrix} \in \mathbb{P}_L$$

Additionally, the vector composed of impulse response is a linear combination of Chebyshev coefficients and polynomial coefficients:

$$u = \begin{bmatrix} h[0] \\ 2h[1] \\ \vdots \\ 2h[L] \end{bmatrix} = W^{-1}q, \text{ where } q = \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_L \end{bmatrix}$$

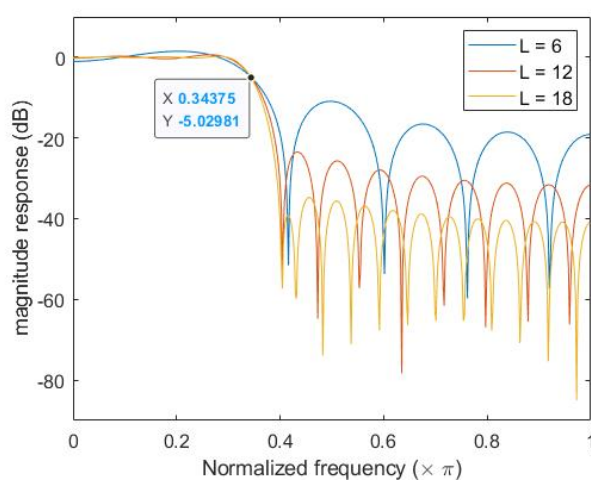
```
function W = myChebyPol(N)
% each column of W represents the n-th Tn(x)
% each row represents the coefficients of x^0, x^1, ..., x^L
% e.g. T1(x) = 1, so the first column is [1, 0, 0, ..., 0]^T
% T2(x) = x, so the second column is [0, 1, 0, ..., 0]^T
W = zeros(N+1, N+1);
% T0(x) term
T0 = zeros(N+1, 1);
T0(1) = 1;
% place T0(x) term into the first column of W
W(:, 1) = T0;
% T1(x) term
T1 = zeros(N+1, 1);
T1(2) = 1;
% place T1(x) term into W
W(:, 2) = T1;
% coef1 is 2x term
coef1 = zeros(N+1, 1);
coef1(2) = 2;
% coef2 is -1 term
% Tn(x) = 2xTn(x-1) - 1Tn(x-2);
coef2 = zeros(N+1, 1);
coef2(1) = -1;
for ii = 3:(N+1)
    tmp = conv(W(:, ii-1), coef1) + conv(W(:, ii-2), coef2);
    W(:, ii) = tmp(1:N+1);
end
end
```



3.

L: relating to filter length (fixed transition width)

Firstly, we take account into different length L . The order of polynomial is directly related to L , which implies a long length filter composed of high-order polynomial. In my perspective, it is equivalent to a long window method in the previous homework. Accordingly, it intuitively might have narrower main-lobe and stronger sidelobe. Moreover, a high-order polynomial is versatile that can bend anywhere if we desire. As a consequence, long length filter would be more like an **ideal filter** with **narrow main-lobe** and **weak sidelobe**. However, in practice, the two conditions can't be sufficed simultaneously, so they are always made a trad-off. Let's continue to justify our inference by the result came from the different L .

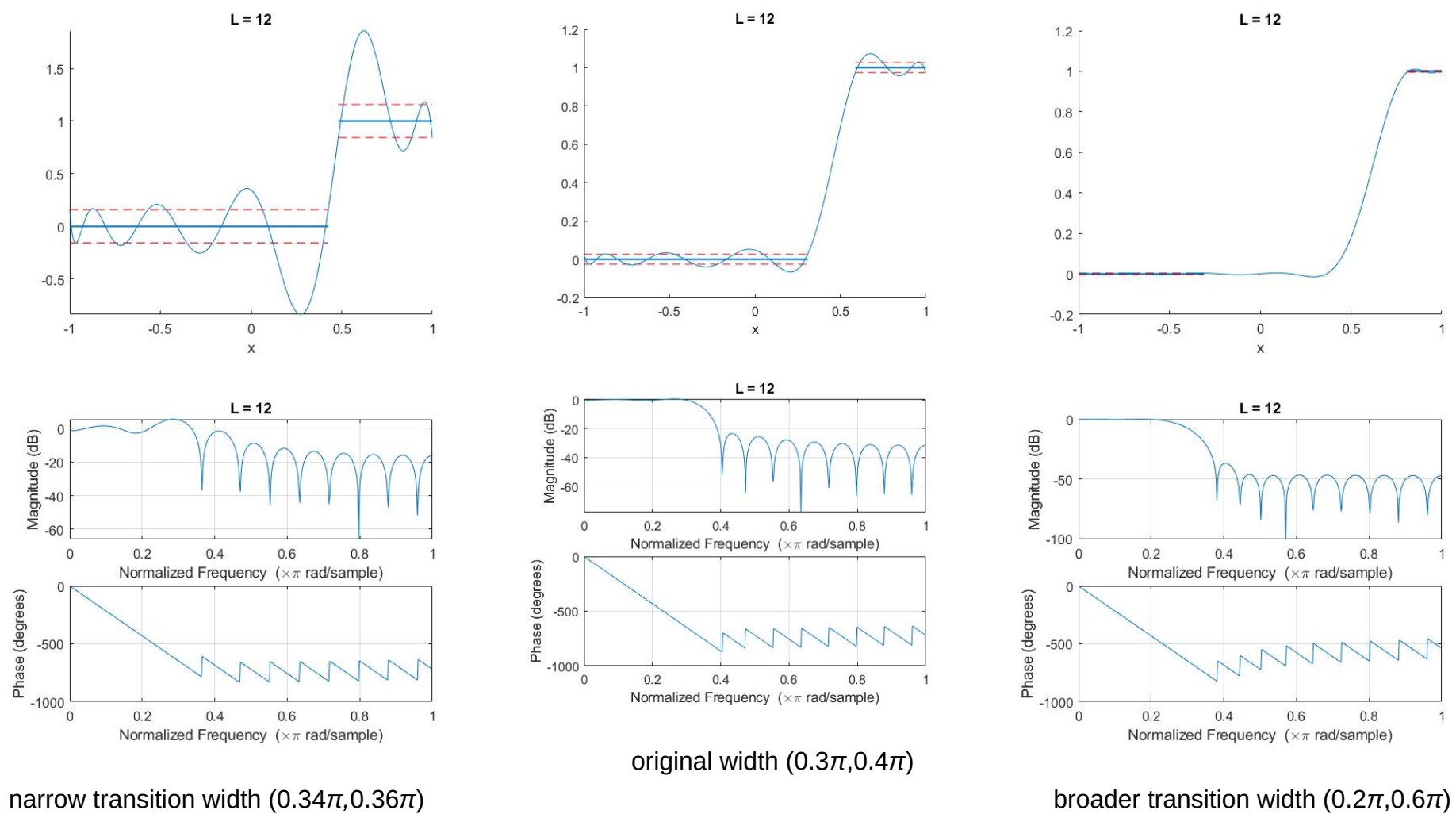


L	tolerance
6	0.1108
12	0.0260
18	0.0090

To my surprised, as the length or the order of polynomial increases, not only main-lobes become narrower but also sidelobes mitigate. It seems to have a potential that would outperform the previous four window-method filters in HW6. Looking for the reason, let's focus on the tolerance. The above table shows that the tolerance becomes strict as the length increases, so it indicates sidelobes are suppressed significantly at the same time. Another reason may be the location of alternating point. Since high-order polynomial has more alternating points in the transition band, it could adjust the curve a little comprehensively. → Really? See the last discussion.

ω_s, ω_p (fixed L)

Secondly, the two parameters are modified to change transition width. Based on the previous homework, transition width is regarded to main-lobe. It becomes narrower so does main-lobe. Let's continue to justify this inference.



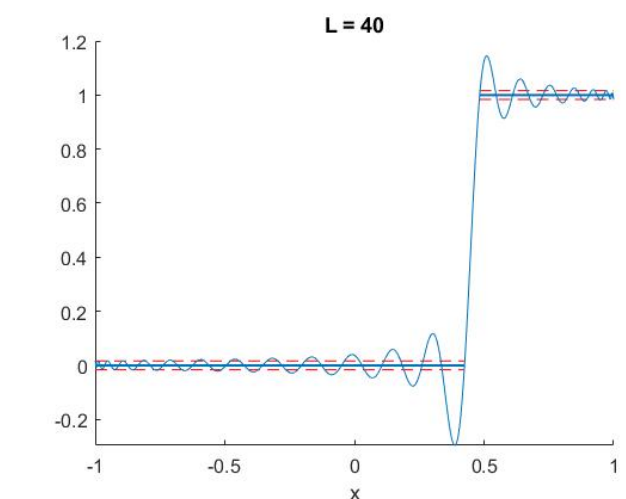
As our expected, the narrowest one also has the narrowest main-lobe among them. It further follows the rule that narrow main-lobe one has strong sidelobe.

Properties

- longer length filter has narrow main-lobe and weak side lobe.
- narrow transition width brings narrow main-lobe and strong side lobe.

In sum, an ideal filter almost does not have transition width and ripples at stopband and passband. However, when transition width is roughly 0.02π , the peak sidelobe grows apparently. Extending filter length is further difficult to suppressed it. For instance, Gilbert's phenomenon is obvious with an increment for a length of 20 as shown in the right figure.

As a result, the designing way still can't break up the rule, that narrow main-lobe filter has strong peak sidelobe.



narrow transition width ($0.34\pi, 0.36\pi$)

Compared with window-method filter.

This method (HW7) is much more complicated, because not only the preprocessing works, such as initializing alternating points and transformation, but further to be optimized. Is hard work really paid off? See the pair of figures below, fig. 2 and fig.3. For the purpose of fair comparison, four filters have the same length, e.g. $L = 50$ in fig.2.

However, the polynomial-method one doesn't pass through -6dB at the cut-off frequency! Some reasons might lead to this circumstances.

- i. **Warning: Matrix is close to singular or badly scaled. Results may be inaccurate.** For a high-order filter, inverse of A seems to inaccurate due to too many micro terms.
- ii. **To suffice the condition of tolerance at passband frequency and stopband frequency.** Transition band has numbers of alternating points such that the curve bends atrociously. For example, L of 21 or 50 samples one shifts and neither passes through -6dB at the cut-off frequency, while L of 20 or 22 samples one exactly goes through that in fig. 2. In details, we directly set the *ind-1-th* alternating point at stopband, which would interrupt the original order seen in fig. 1. It gives rise to the curve bending in advance during transition band.

Since the polynomial method has not be optimized yet, it might be improved after optimization. Another conclusion after improving the initial way for alternating points, this design method seems not to guarantee that magnitude response falls off -6dB at $\omega_c = (\omega_s + \omega_p)/2$ whereas window method does.

Any better way to initialize alternating points?

Due to the above discussion ii, we suggest trying to limit the number of alternating points during transition band. Namely, avoid interrupting the order of x. A preliminary thought is dropping out alternating points located at transition band. For instance, in fig. 1, the 39-th to 41-th points in the red block will be dropped out and compensated with new points generated randomly. See in the following snippet.

```
rng(7414)
....
xx = cos(ff); xx = xx(:);
% Change the one closest to x_p to exactly x_p
% -----original code -----
[val_p,ind] = min(abs(xx-x_p));
xx(ind) = x_p;
% -----
% we further deal with the ind-1-th point to avoid interrupting x
% in transition band
[val_s,ind_s] = min(abs(xx-x_s)); % find the stopband point
% it should belong to the right index rather than the ind-1-th
xx(ind_s) = x_s;
% find points in the transition band
xx_drop = xx(ind_s+1:ind-1);
% determine whether any points are requires to be dropped out.
while xx_drop ~ []
    % random generate compensated alternating points
    xx_in = 2*rand(length(xx_drop),1) - 1
    % place it in xx array in sequence
    xx_in = [xx(1:ind_s);xx(ind:end);xx_in];
    xx = sort(xx_in,'ascend');
    % since random generated point may still in the transition band
    % check whether any points are requires to be dropped out once again
    % until suffice the condition
    [val_p,ind] = min(abs(xx-x_p));
    xx(ind) = x_p;
    [val_s,ind_s] = min(abs(xx-x_s));
    xx(ind_s) = x_s;
    xx_drop = xx(ind_s+1:ind-1);
end
```

After improvement, the polynomial one passes through -6dB at cut-off frequency seen in fig. 4. In addition, it outperforms other window methods! Note that a drawback of this way is hidden in randomly generating. Modifying random seed would still give rise to shifting at cut-off frequency slightly, e.g. rng(97414).

xx	
52x1 double	
	1
19	-0.4457
20	-0.3898
21	-0.3324
22	-0.2737
23	-0.2139
24	-0.1534
25	-0.0923
26	-0.0308
27	0.0308
28	0.0923
29	0.1534
30	0.2139
31	0.2737
32	0.3324
33	0.3898
34	0.4457
35	0.5000
36	0.5524
37	0.6026
38	0.6506
39	0.6961
40	0.7390
41	0.6374
42	0.8090
43	0.8502
44	0.8810
45	0.9085
46	0.9325
47	0.9529
48	0.9698
49	0.9830
50	0.9924
51	0.9981

xx	
14x1 double	
	1
1	-1
2	-0.9709
3	-0.8855
4	-0.7485
5	-0.5681
6	-0.3546
7	-0.1205
8	0.1205
9	0.3546
10	0.6374
11	0.8090
12	0.8855
13	0.9709
14	1

fig.1 Why magnitude response shift.

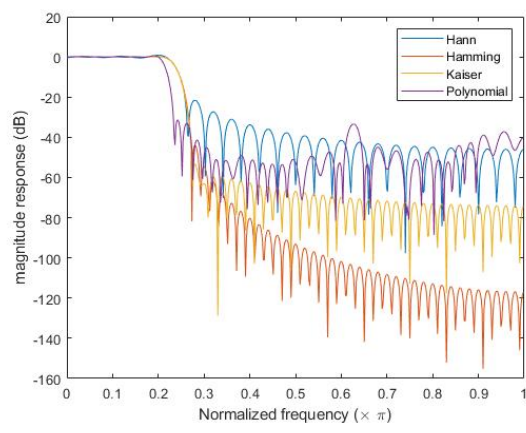


fig.2 $L = 50$ for each filter

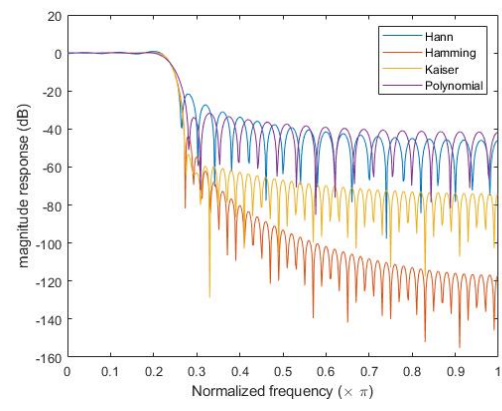


fig.3 $L = 22$ for polynomial one
otherwise $L = 50$

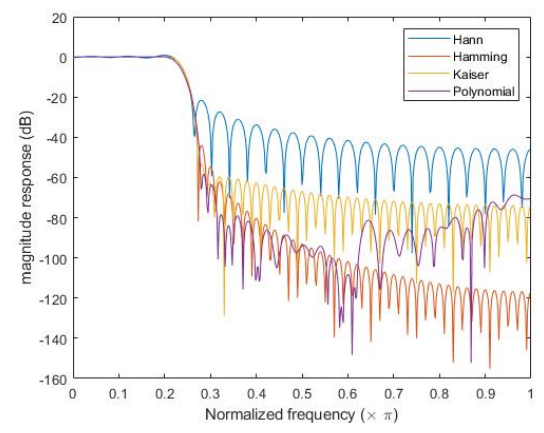


fig.4 improved version,
 $L = 50$ for each filter